PRE-LAB 05 — Interaction is more than drawing circles

15–20 minutes · Complete before class · D3 v7

Goal: Run one complete interactive chart, understand what happens when you hover, then inspect a small CSV. The examples use **fictional study spaces**, not the COVID-19 dataset or final solution.

1. Run a working example (3 min)

Open the `examples/` folder in VS Code. Start a local web server:

```
cd examples
python -m http.server 8000
```

Visit **http://localhost:8000/01_worked_interaction.html**. You need Internet access to load D3 v7. Move the mouse over a circle. You should see a tooltip with the name and number of visitors; the circle gets a highlight. Move away, and the tooltip disappears. Open **DevTools → Console** and check that 5 rows produce 5 circles.

2. Read the complete HTML example (6 min)

Find the lettered sections **A–E** in `01_worked_interaction.html`. You do **not** need to type the code again.

Section	What to look for	Your question
A — Model	Each object stores `x`, `y`, `visitors`	What controls circle position vs size?
B — Canvas	`scaleLinear`, `scaleSqrt`, SVG axes	Why is Y range reversed?
C — Tooltip	`showTooltip`, `moveTooltip`, `hideTooltip`	Why do we need all three?
D — Marks	`.data(...).join('circle')` and `.on(...)`	What are `event` and `d`?
E — Check	Compare array length with circles	What number do you expect?

The key event pattern:

```
plot.selectAll('circle.spot')
  .data(spaces, d => d.id)
  .join('circle')
  // ... attributes that position and size each circle ...
  .on('pointerenter', showTooltip)   // Show content on arrival
  .on('pointermove', moveTooltip)   // Follow the pointer
  .on('pointerleave', hideTooltip); // Clean up on departure
```

The **complete implementation**, including tooltip HTML, positioning, and styles, is in the runnable file. In D3 v7 a callback receives **`(event, d)`**: `event` is the pointer event; `d` is the datum bound to that circle. Older book examples may use `d3.event` or `.on('mouseover', function(d){...})`; those are not the v7 signature.

3. Make 3 small edits (5 min)

Edit `01_worked_interaction.html`, save and refresh after each change:

1. Change **Media Room visitors** from `28` to `55`. *Predict:* what changes in size and tooltip?
2. Change **Quiet Study x** from `53` to `83`. *Predict:* where does the circle move? Why?
3. Comment out `.on('pointerleave', hideTooltip)` and move away from a circle. *Observe:* why does the tooltip remain? Restore this line afterward.

Write down **one expected result → one observed result** for each edit.

4. Inspect numeric CSV fields (3–4 min)

Open **http://localhost:8000/02_inspect_practice_csv.html** with the server still running. Compare the raw table with the filtered table in DevTools. The practice CSV has **5 rows, of which 4 are valid**; one contains a deliberately invalid coordinate. Explain how the code turns string columns into numbers and excludes a malformed row.

In Lab 05, repeat this inspection on the **Johns Hopkins COVID-19 time-series CSV**. Its date columns use **US `M/D/YY` formatting**, whereas the lab's Vietnamese-format **05/04/2020 means 5 April 2020**, corresponding to **`4/5/20`**.

Bring 3 answers to class

1. How are **`event`** and **`d`** different in a D3 event handler?
2. Why do we need to hide or reposition a tooltip?
3. What evidence proves every valid data row has a visual mark?

Pre-lab boundary: The worked example demonstrates interaction on fictional data. During Lab 05, your pair must independently choose the representation of the real historical COVID dataset, test it and explain it.